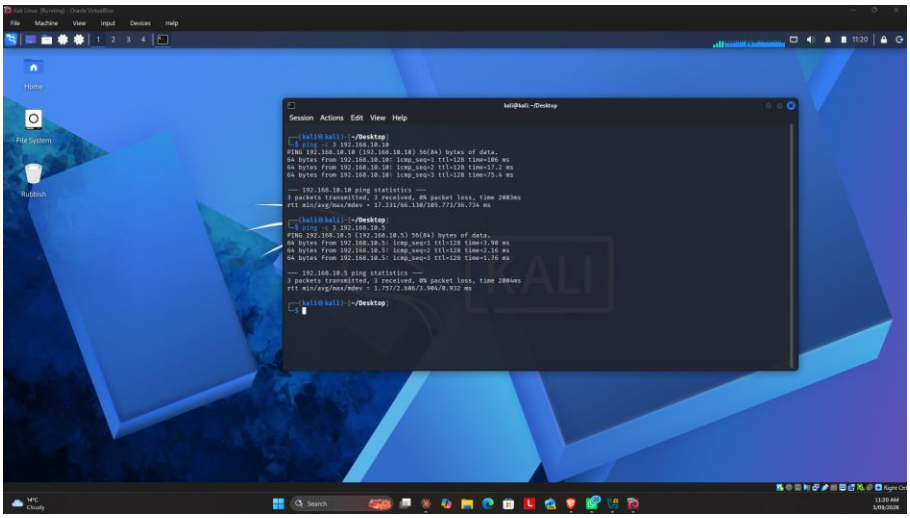
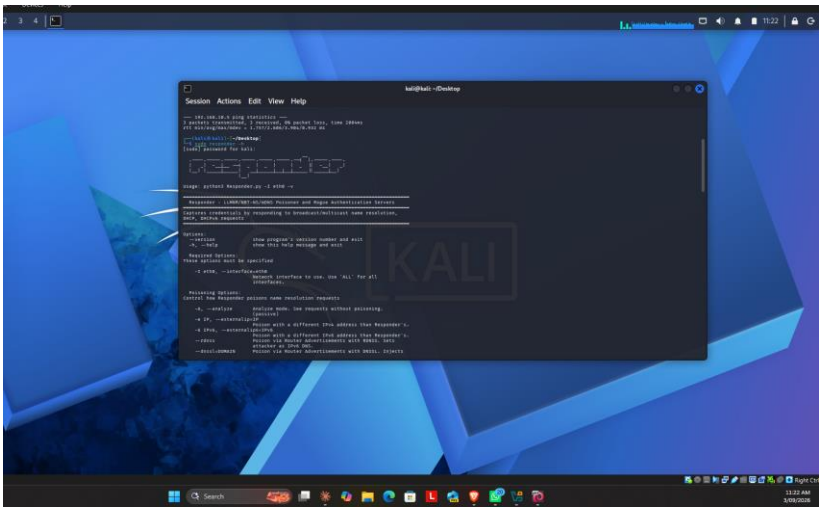
Active Directory attack lab

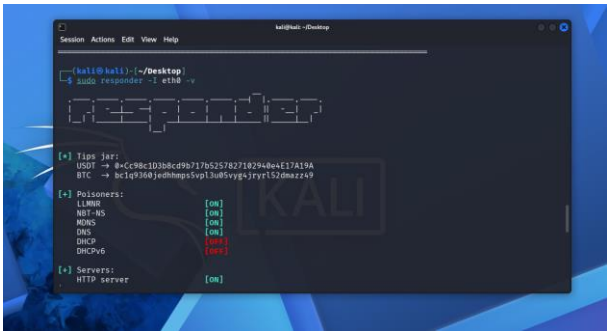
1) First, we have opened all three machines which are Kali, Win server 2022, and Win 11. And then, checked using ping from kali machine.



2) Confirmed Responder (LLMNR/NBT-NS poisoning tool) is pre-installed on Kali — checked via responder -h before starting the attack. **Responder** is an open-source penetration testing tool that listens on a local network and responds to name-resolution broadcast requests (LLMNR, NBT-NS, and mDNS) sent by Windows systems, impersonating the requested host to capture authentication credentials sent by the victim machine.

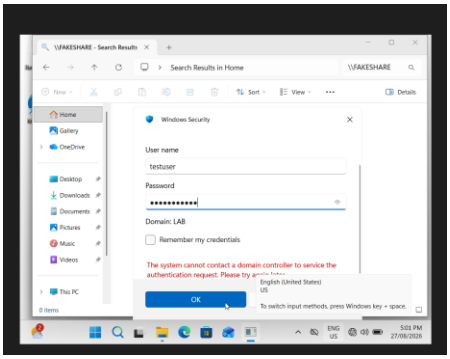


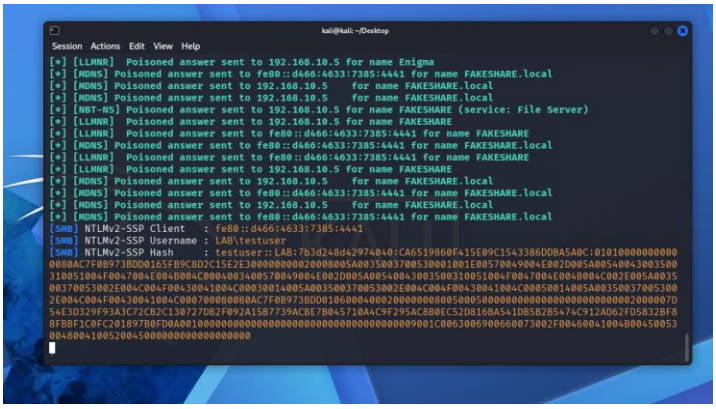
3) Started Responder in listening mode on Kali's network interface (eth0) — now waiting to capture any Windows name-resolution requests on the LabNet subnet.

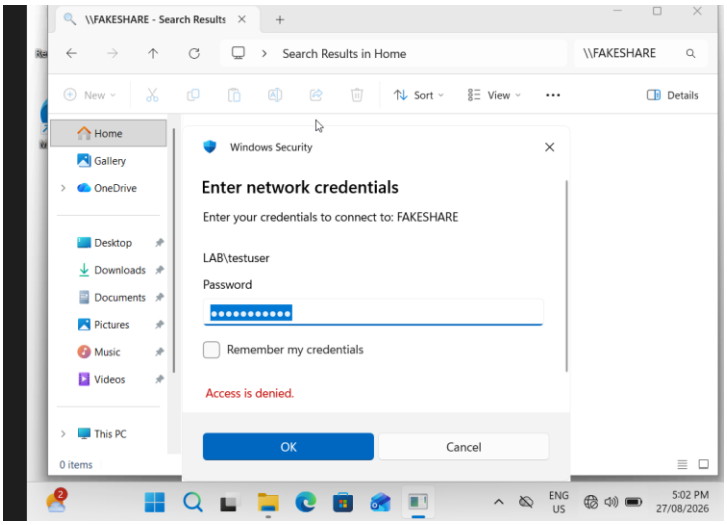


4) Triggered a failed network lookup on Windows 11 (\FAKESHARE) to generate an LLMNR broadcast for Responder to intercept. **LLMNR** = Link-Local Multicast Name Resolution. In plain terms: it's a backup way for Windows computers on the same local network to find each other by name when normal DNS fails — by shouting the request out to everyone nearby instead of asking for a proper server. That "shouting to everyone" behavior is exactly the weakness of Responder exploits.

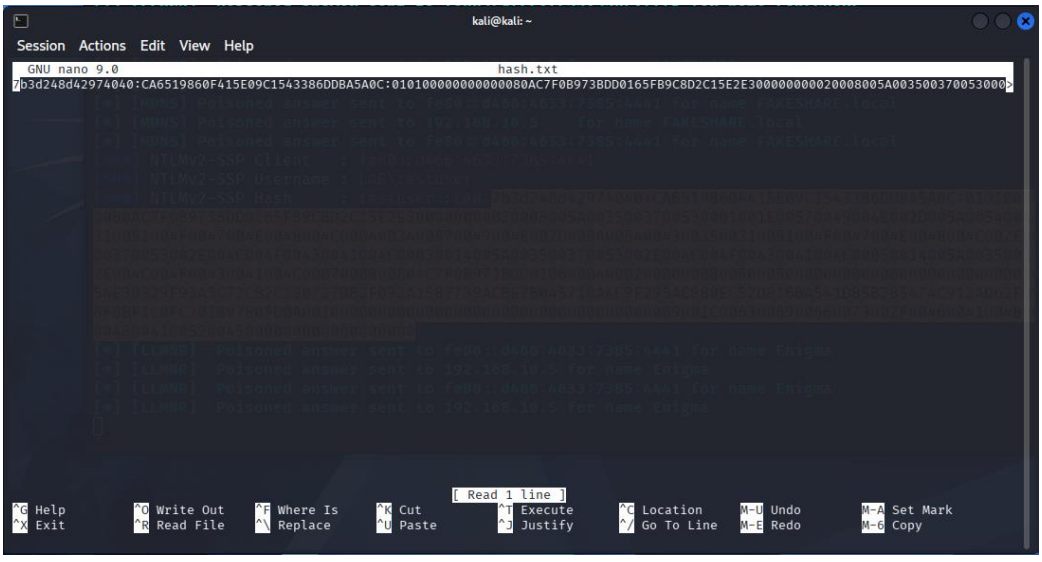
"Triggered the LLMNR/NBT-NS poisoning attack end-to-end: entered fake credentials (testuser / a fake password) into the Windows Security login prompt for the non-existent \FAKESHARE network location. Windows 11 attempted to authenticate using those credentials — and in doing so, sent an NTLMv2 authentication hash across the network. Responder, running on Kali and impersonating FAKESHARE, intercepted and displayed the full NTLMv2-SSP hash (LAB\testuser) in real time. Back on Windows 11, the login attempt itself failed as expected — first with 'Access is denied,' then with 'the system cannot contact a domain controller' — since testuser was never a real account. Those failures were intentional and don't matter: the actual goal of the attack, capturing the password hash in transit, had already succeeded before Windows even rejected the login."



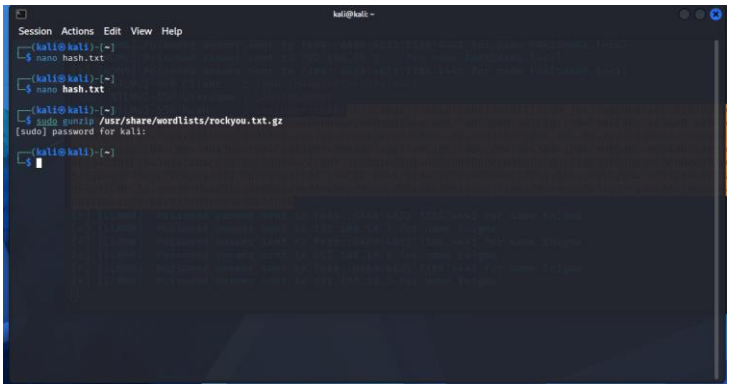




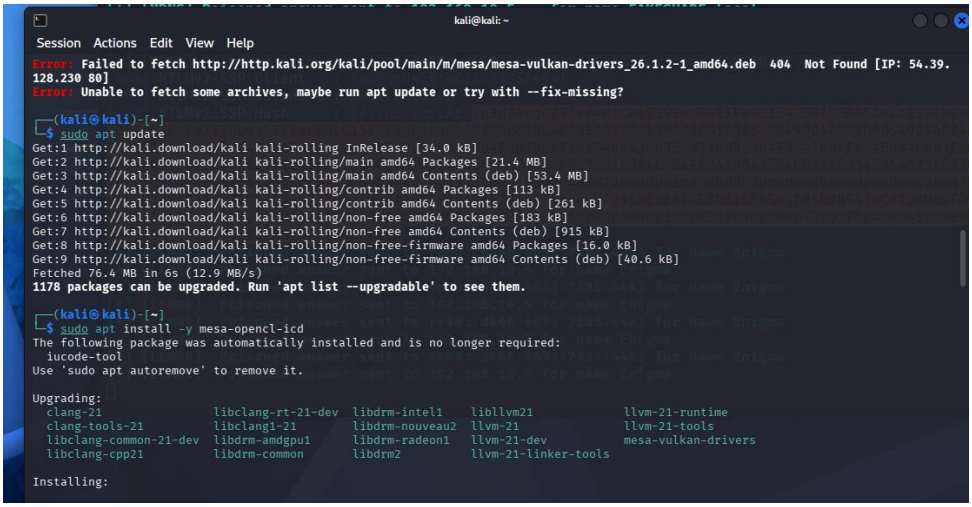
5) Saved the captured NTLMv2 hash to a text file (hash.txt) to prepare it for cracking with Hashcat.



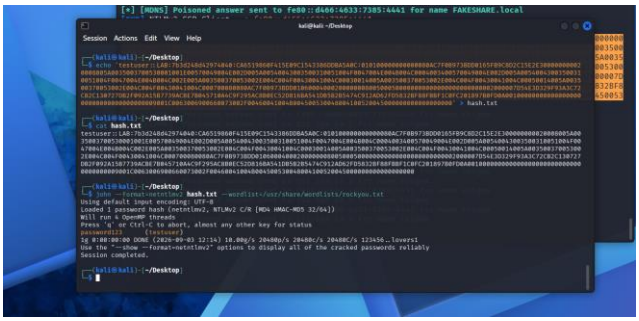
6) Unzipped Kali's built-in rockyou.txt wordlist (a list of ~14 million real leaked passwords) to use as the dictionary for cracking the captured NTLMv2 hash with Hashcat.



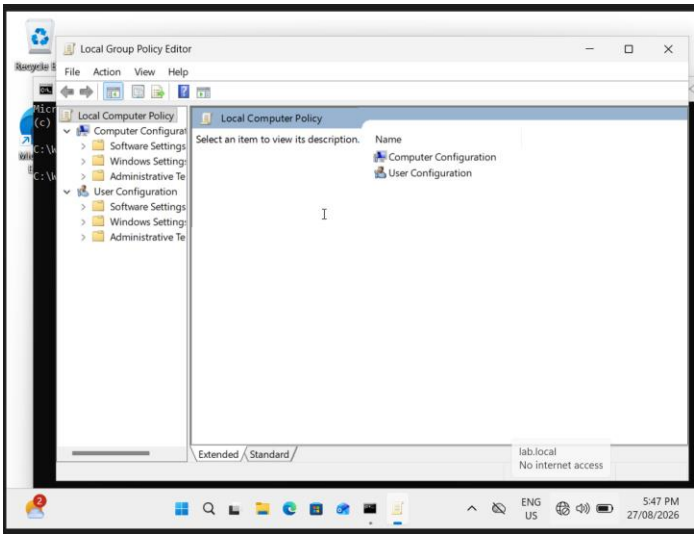
7) Attempted to crack the captured NTLMv2 hash using Hashcat, but it failed immediately with 'No OpenCL, HIP or CUDA compatible platform found' — Hashcat needs a GPU driver to run, and this VirtualBox VM has no real graphics card. Tried installing a CPU-based OpenCL driver (pocl-opencl-icd) as a workaround, but that exact package wasn't available in Kali's repository. Searched for alternatives and found mesa-opencl-icd, a software-based OpenCL driver that works without a real GPU. The first install attempt failed with a 404 error due to a temporarily out-of-sync package mirror; running sudo apt update to refresh the package list fixed that, and mesa-opencl-icd installed successfully on the second attempt — giving Hashcat a working CPU-based OpenCL platform to run on.



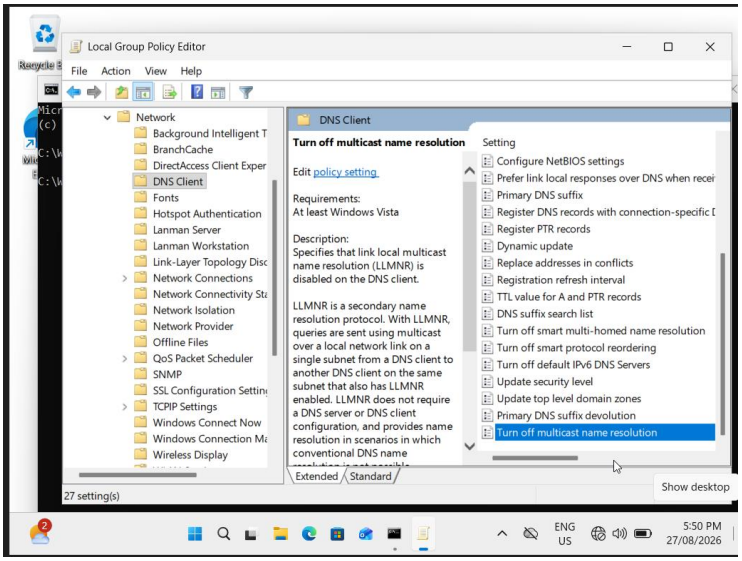
8) "Initially tried to crack the captured NTLMv2 hash using Hashcat, but it failed because Hashcat requires GPU acceleration (OpenCL/CUDA) to run, and this VirtualBox VM has no real graphics card. Tried installing a CPU-based OpenCL driver (mesa-opencl-icd) as a workaround, but Hashcat still reported 'No devices found' — the software driver wasn't exposing a usable compute device inside the VM. Rather than keep fighting GPU driver issues for a single hash, switched to John the Ripper instead, which cracks hashes purely on the CPU with no graphics driver required at all. Also had to fix the hash file first — an earlier copy-paste had accidentally cut off the username and domain portion of the hash. Once the full hash was in place, John the Ripper cracked it in under 1 second against the rockyou.txt wordlist, recovering the password 'password123' and completing the attack from network poisoning to plaintext password."



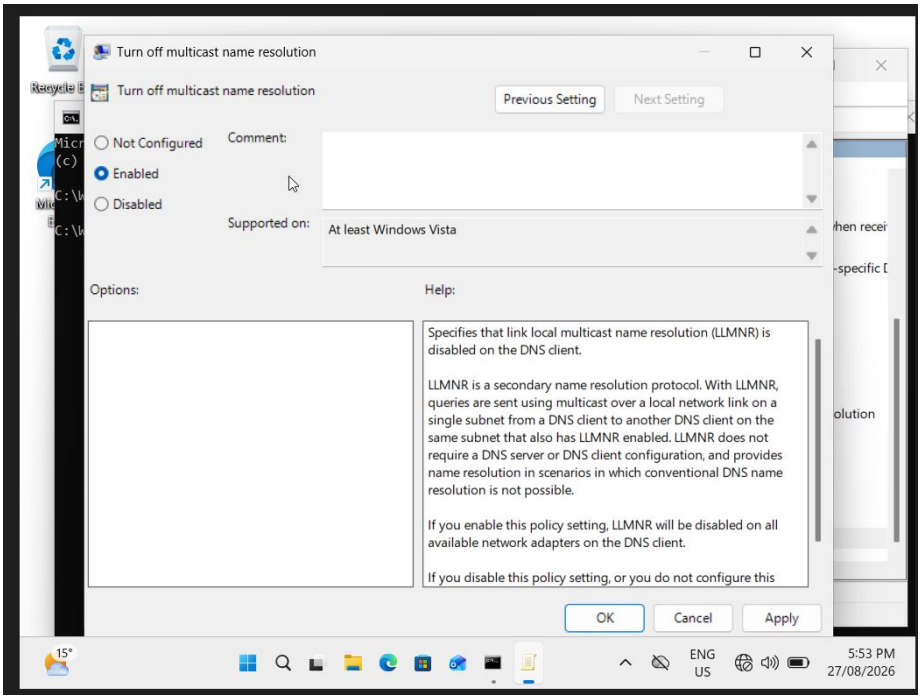
9) Opened the Local Group Policy Editor on Windows 11 to disable LLMNR as a defensive fix against the poisoning attack.



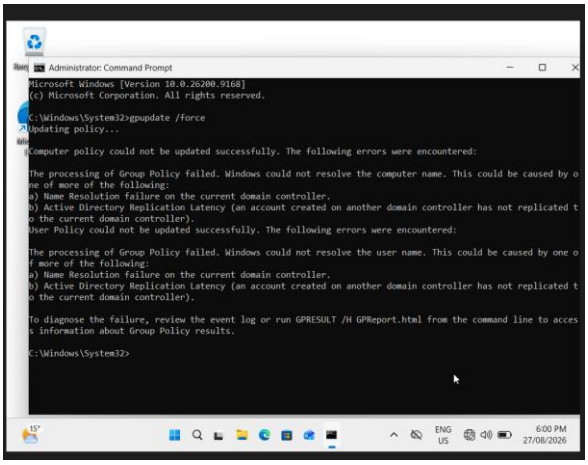
10) Opened the DNS Client settings folder in Group Policy — this is where the setting to disable LLMNR lives.



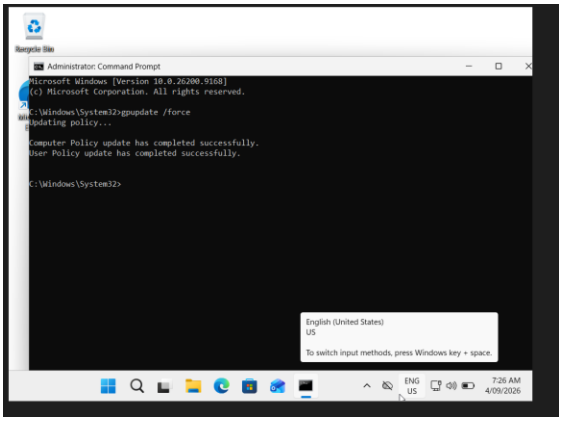
11) Set 'Turn off multicast name resolution' to Enabled in Group Policy — this disables LLMNR on Windows 11, closing the vulnerability the earlier attack exploited.



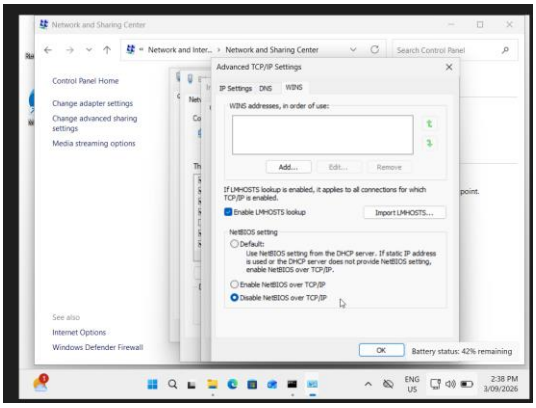
12) Ran gpupdate /force to apply the new LLMNR-disabling policy immediately but hit a real network error — Windows 11 couldn't reach the domain controller to complete a full policy refresh (name resolution / replication error). This is a domain-communication issue, separate from the local Group Policy change we made in gpedit.



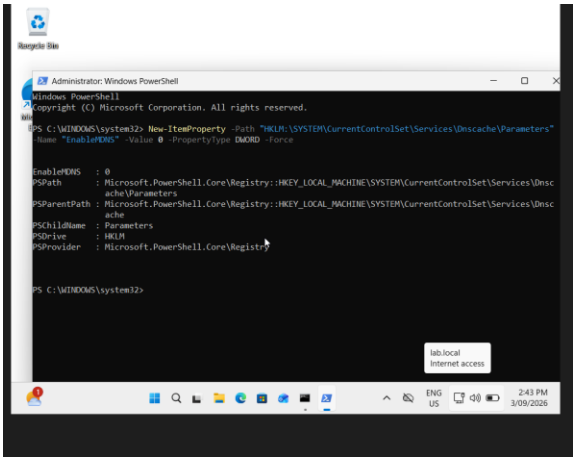
13) Group Policy finally applied successfully — after restarting Windows 11 to clear stale Kerberos tickets and re-syncing the clock with the domain controller, `gpupdate /force` completed with no errors: "Computer Policy update has completed successfully. User Policy update has completed successfully." This confirms the LLMNR-disabling setting from Local Group Policy is now genuinely enforced by the domain, not just sitting unconfirmed. The real troubleshooting story here: an unexpected VM shutdown threw the two machines' clocks 8 days out of sync, which broke Kerberos authentication (LDAP Bind errors) and cascaded into a DNS/firewall network-profile issue — all resolved by re-syncing the clock, fixing the server's network category, and a clean restart.



14) Disabling NBT-NS (NetBIOS Name Service) on Windows 11's network adapter, via Advanced TCP/IP Settings → WINS tab. This was necessary because our first defense test showed disabling LLMNR alone wasn't enough — when LLMNR got no answer, Windows automatically fell back to NBT-NS and mDNS, two older, separate name-resolution protocols that Responder also listens for, and a hash was still captured. NBT-NS is a legacy protocol from Windows NT-era networking, and turning it off here removes one of the two remaining fallback paths for this attack.



15) Disabled mDNS (multicast DNS) on Windows 11 using a PowerShell command that sets the `EnableMDNS` registry value to 0 under the DNS Client service's settings. There's no checkbox for this in any Windows settings menu — it can only be turned off through the registry. Output confirms `EnableMDNS : 0`, meaning the change was applied successfully. With LLMNR (via Group Policy), NBT-NS (via the network adapter), and now mDNS all disabled, all three of Windows' automatic name-resolution fallback methods are shut off — closing the paths our earlier attack used to succeed.



16) Final defense verification — LLMNR/NBT-NS/mDNS poisoning attack fully blocked.

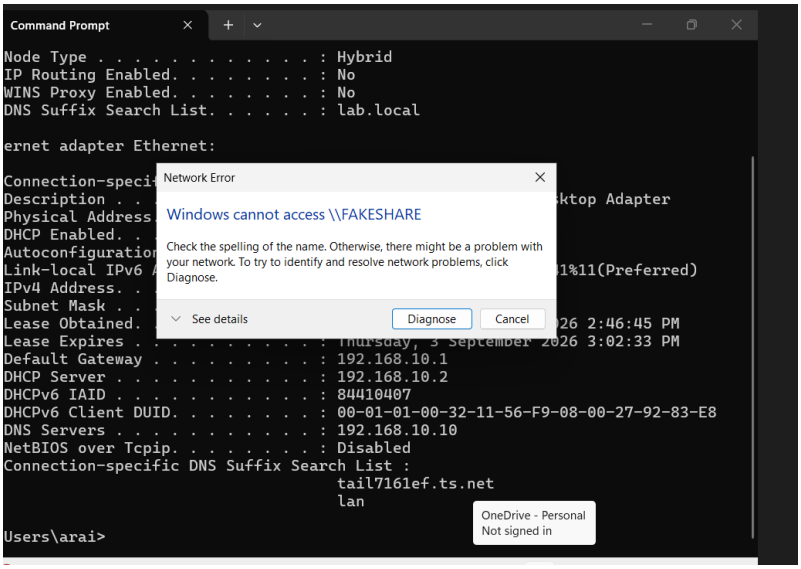
After capturing and cracking a real NTLMv2 password hash earlier using Responder (via LLMNR poisoning), the goal was to prove the attack could actually be stopped — not just talk about the theory.

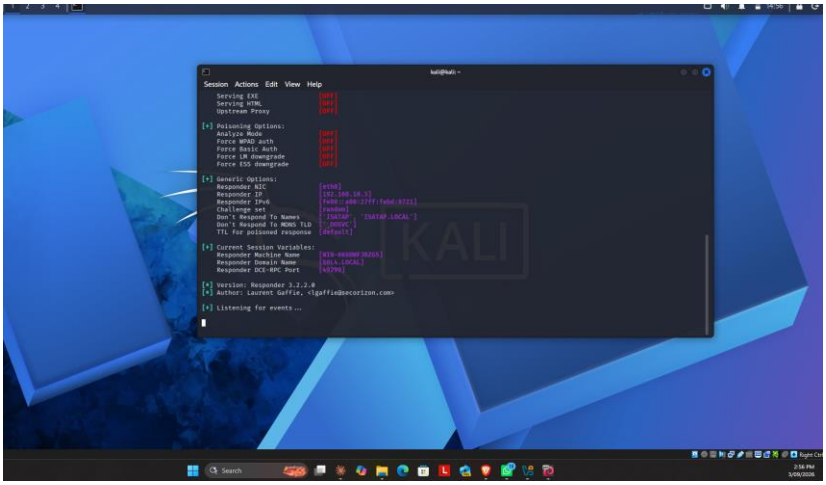
The first defense attempt only disabled LLMNR through Group Policy. Retesting the attack showed it still worked — Windows automatically fell back to two older name-resolution methods, NBT-NS and mDNS, and Responder captured a hash through those instead. This showed that disabling LLMNR alone isn't a real fix, which is an important, realistic finding on its own.

To fully close the gap, two more settings were disabled: NBT-NS (through the network adapter's advanced TCP/IP settings) and mDNS (through a registry change, since Windows has no menu option for it). Along the way, NBT-NS briefly reset itself after a reboot due to the DHCP lease renewing, so it had to be reapplied and verified.

With all three protocols disabled, the attack was run one final time: Responder was left listening on Kali, and the same fake network share (\\FAKESHARE) was triggered from Windows 11. This time, Windows 11 returned "Windows cannot access \FAKESHARE" and Responder's terminal stayed completely silent — no poisoned answers, no captured credentials.

This proves the full defense works end-to-end: an attacker on the same network can no longer trick this machine into leaking a password hash through any of Windows' automatic name-resolution fallback methods.


Command Prompt
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : lab.local
ernet adapter Ethernet:
Network Error
Windows cannot access \\FAKESHARE
Check the spelling of the name. Otherwise, there might be a problem with your network. To try to identify and resolve network problems, click Diagnose.
See details
Diagnose
Cancel
Lease Expires . . . . . : Thursday, 3 September 2026 3:02:33 PM
Default Gateway . . . . . : 192.168.10.1
DHCP Server . . . . . : 192.168.10.2
DHCPv6 IAID . . . . . : 84410407
DHCPv6 Client DUID. . . . . : 00-01-01-00-32-11-56-F9-08-00-27-92-83-E8
DNS Servers . . . . . : 192.168.10.10
NetBIOS over Tcpip. . . . . : Disabled
Connection-specific DNS Suffix Search List :
tail7161ef.ts.net
lan
OneDrive - Personal
Not signed in
Users\arai>


Session Actions Edit View Help
KALI